숨결톡톡 | RESONANCE ANALYSIS

로그 스케일 적용 및 공진 후보 탐색 알고리즘

수식 검증 · 주파수 보정 · Arduino IDE 전체 소스코드

이번 개정에서 바뀌는 것

원래 그래프	X=Frequency(Hz), Y=Acceleration(m/s^2) 유지
로그 계산	가속도 dB와 속도 보정 dB를 따로 출력
후보 판정	속도 환산값의 내부 피크만 예비 후보로 보고
구동 방식	기존 구동부는 변경하지 않음; 이 소스에는 구동부 미포함

> Kt 추가 개정
> 첨부 필기의 시간·주파수 식 검토 + 코일 전류 수동 입력 + 전체 코드 교체

핵심: 로그 변환 자체는 공진 위치를 바꾸지 않음.

속도 환산과 내부 피크 판정을 결합하되, 입력 가진력 미측정 한계는 남음.

작성일 2026.11.03 | 센서 BNO085 | Arduino IDE (ESP32)

DESIGN NOTE

01. 로그 스케일의 의미

로그 표시와 물리적 주파수 보정을 반드시 구분한다.

A. 로그만 적용하면

현재 가속도 RMS를 그대로 두고 Y축만 dB로 바꾸는 방법이다. 기준 가속도를 1 m/s^2 로 명시한다.

$$L_a(f) = 20\log_{10}\left(\frac{A_{\mathrm{RMS}}(f)}{A_{\mathrm{ref}}}\right)$$

로그 함수는 양수 구간에서 단조 증가한다. 따라서 원래 가속도가 45 Hz에서 최대라면, 로그 가속도도 45 Hz에서 최대다.

B. 속도 환산은 별도의 물리 계산

$$A(f) = (2\pi f)^2 X(f)$$

정현파 변위의 두 번째 시간미분이 가속도이므로, 같은 변위에서는 주파수의 제곱이 가속도 진폭에 곱해진다.

$$V_{\mathrm{RMS}}(f) = \frac{A_{\mathrm{RMS}}(f)}{2\pi f}$$

구동 주파수 성분의 가속도 RMS를 각주파수로 나누면 속도 RMS가 된다. 기존 센서를 유지할 수 있는 부분적 보완이다.

> 주의: 가속도에서 속도로 환산해도 입력 가진력의 주파수 의존성은 제거되지 않는다. 따라서 자동 판정 문구는 반드시 예비 공진 후보로 제한한다.

02. 로그 보정식과 기준값

30 Hz는 표시용 기준 주파수이며 임상·기계적 정답이 아니다.

가속도 레벨 vs 속도 레벨

$$L_v(f) = 20\log_{10}\left(\frac{V_{\mathrm{RMS}}(f)}{V_{\mathrm{ref}}}\right)$$

여기서 Vref = 0.001 m/s(1 mm/s)이다. 속도 dB는 가속도 dB와 다른 참조 단위를 사용하므로 절대 dB 수치를 서로 비교하면 안 된다.

기준 주파수를 이용한 보정 레벨

$$L_{\mathrm{corr}}(f) = L_a(f) - 20\log_{10}\left(\frac{f}{f_{\mathrm{ref}}}\right)$$

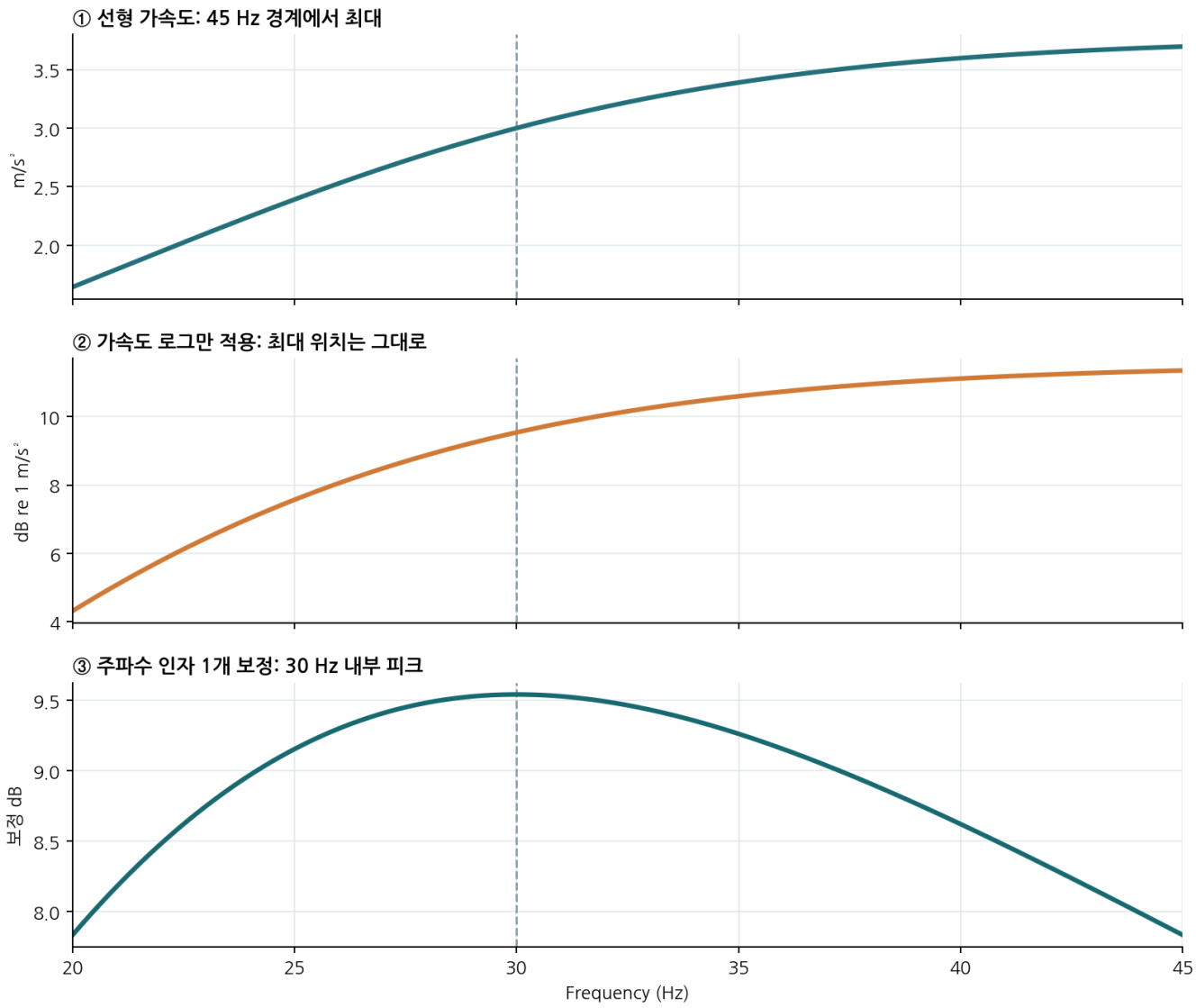
fref = 30 Hz를 기준으로 정하면, 높은 주파수일수록 20 log10(f/30) 만큼 가속도 레벨에서 차감한다. 이는 속도 환산값과 같은 주파수별 순위를 만든다.

주파수	30/f 보정계수	dB 차감량
20 Hz	1.5000	+3.522 dB
25 Hz	1.2000	+1.584 dB
30 Hz	1.0000	0 dB
35 Hz	0.8571	−1.339 dB
40 Hz	0.7500	−2.499 dB
45 Hz	0.6667	−3.522 dB

> 핵심: 30/f는 실험 보정계수가 아니라 수학적 환산계수다. 보정 dB와 속도 dB는 같은 피크 위치를 주며, 두 지표를 독립 검증으로 착각해서는 안 된다.

03. 로그 비교 그래프: 3가지 방식

이론적 MSD 예제 / 실제 흉벽 측정 데이터가 아님



가정: 1자유도, 일정한 정현파 가진력, 무감쇠 고유진동수 30 Hz, 감쇠비 0.6. 이 가상 예에서 가속도와 그 로그는 끝점 최대, 속도로 환산한 보정 dB는 30 Hz의 내부 피크를 보인다.

> 로그 표시만 적용하는 ②는 추천 문제를 해결하지 못한다. ③은 물리적 보정이지만 실제 장치의 가진력 변화까지 보정하지 못한다.

DESIGN NOTE

04. 기존 Arduino 프로젝트에서 달라지는 점

기존 DATA 전송은 유지하고 별도 LOG 출력을 추가했다.

명령	기능
M,30	외부 구동부가 30 Hz로 동작 중일 때 계측
L	측정한 모든 주파수의 선형·로그 지표 출력
S	속도 환산값의 내부 피크를 예비 후보로 판정
C	저장된 주파수 데이터 초기화
X	센서 측정만 중단; 가진기는 별도 정지 필요

기존 웹 파서를 보호하는 출력 형식

DATA 행은 기존 형식(주파수, 가속도 RMS, 변위 RMS, 표본 수, 실효 샘플률, R^2, 품질)을 그대로 유지한다. LOG 행은 사용자가 L 명령을 입력했을 때만 출력한다.

> LOG,Hz,AccelRMS,Accel_dB,VelocityRMS_mmps,Velocity_dB,Corrected_dB,Trials

웹에서 로그로 표시하려면

웹 소스코드가 제공되지 않았으므로 여기서 웹 코드를 직접 수정할 수는 없다. 웹은 기존 DATA의 주파수와 가속도 RMS로 아래 값을 자체 계산하여 Y축을 로그 보조 보기로 표시할 수 있다.

```
const aDb = 20 * Math.log10(accelRms / 1.0);        // dB re 1 m/s²
const vRms = accelRms / (2 * Math.PI * frequencyHz); // m/s
const vDb = 20 * Math.log10(vRms / 0.001);          // dB re 1 mm/s
const corrDb = aDb - 20 * Math.log10(frequencyHz / 30);
```

원래 X축 주파수(Hz)는 그대로 선형으로 둔다. 20~45 Hz처럼 좁은 주파수 범위에서는 X축 로그 변환의 이점이 작다.

DESIGN NOTE

05. 실측값 대입, 판정 기준 및 한계

45 Hz 실제 값 1개만 계산하며 다른 주파수의 측정값은 임의로 만들지 않는다.

45 Hz, A = 6.307 m/s²

값	계산 결과
가속도 레벨	15.996 dB re 1 m/s²
속도 환산 RMS	22.306 mm/s
속도 레벨	26.969 dB re 1 mm/s
30 Hz 기준 보정 레벨	12.475 dB

> 이 숫자는 45 Hz 한 점을 변환한 결과다. 다른 주파수에서 변환된 속도가 더 클 수도 작을 수도 있으므로 공진 후보는 전 구간 데이터로만 판단한다.

추천 기준 및 실패 상태

① 20~45 Hz 스윕의 각 측정 주파수마다 안정화 후 가속도 기본파 RMS를 구한다.
② 속도 RMS 및 보정 dB를 계산한다.
③ 내부 주파수에서 양옆보다 높은 경우만 후보로 보고한다.
④ 20 Hz 또는 45 Hz의 경계 최대는 확정하지 않는다.
⑤ 반복 측정과 입력 가진력 검증 전에는 결론을 “예비 후보”로 제한한다.

아직 해결되지 않은 부분

트랜스듀서 코일 전류와 실제 접촉 전달력은 이 코드에서 측정하지 않는다. 제조사 공칭 공진주파수 또는 임의의 가진력 상수 Kt를 대입해도 개인별 흉벽 공진이 자동으로 보정되지는 않는다. 추가적인 전류·힘 정규화 없이 복합 결합계의 고유진동수를 확정할 수 없다.

$$F_0(f) = K_t I_{coil}(f)$$

위 관계는 코일 전류를 독립적으로 측정할 때 활용할 수 있다. 현재 INA226의 앰프 공급 전류는 코일 교류 전류 RMS가 아니다.

07. Kt 유도 검토: 시간 영역

첨부 필기의 유도식은 조건부로 타당하다. 실제 Kt 숫자는 아직 주어지지 않았다.

> 현재 확인된 것: 유도식과 단위.
> 아직 필요한 것: 같은 좌표의 m, c, k 및 같은 주파수의 코일 전류·가속도.

선형 1자유도 모델의 출발점

$$F_{\mathrm{em}}(t) = K_t i(t), \qquad [K_t] = \mathrm{N/A}$$

$$m\ddot{x}(t) + c\dot{x}(t) + kx(t) = K_t i(t)$$

m: 유효질량 [kg] · c: 점성감쇠계수 [N·s/m] · k: 강성 [N/m]
x: 정의한 동일 기계 좌표의 변위 [m] · i: 해당 코일 전류 [A]
중력·정적 예압은 평형점에서 제거한 소진동 좌표를 가정한다.

순간값의 비율과 최소제곱 추정

$$K_t = \frac{m\ddot{x} + c\dot{x} + kx}{i} \qquad (i \neq 0)$$

$$\widehat{K_t} = \frac{\sum_n i_n (m a_n + c v_n + k x_n)}{\sum_n i_n^2}$$

필기의 최소제곱식도 맞다. 다만 m,c,k를 알고 있고, 전류와 운동 데이터가 시간 동기화되어야 한다. 전류가 0에 가까운 순간의 단순 나눗셈은 불안정하다. 전류 측정 오차·모델 오차가 있으면 최소제곱도 편향될 수 있다.

> 이번 코드는 위 시간영역 최소제곱을 실행하지 않는다. 기존 BNO085 기본파 RMS 추출에 맞춰 다음 페이지의 주파수 영역 크기 식을 구현했다.

08. 주파수 영역: RMS 식으로 연결

정현파 정상상태 · 1자유도 · 선형 모델 · 같은 기계 좌표가 전제다.

라플라스와 복소 진폭

$$\frac{X(s)}{I(s)} = \frac{K_t}{ms^2 + cs + k} \quad \text{(zero IC)}$$

$$K_t = (k - m\omega^2 + jc\omega)\frac{\tilde{X}}{\tilde{I}}, \quad \omega = 2\pi f$$

라플라스 전달함수는 초기조건 0에서 성립한다. 복소 진폭식은 정상상태 정현파 응답에 적용한다. 진폭의 크기만 사용하면 |Kt|만 얻으며 부호·위상 일치는 검증할 수 없다.

변위 진폭을 가속도 RMS로 바꾸기

$$a_{\text{rms}} = \omega^2 x_{\text{rms}}$$

$$|\widehat{K_t}| = \frac{a_{\text{rms}}}{I_{\text{coil, rms}}\omega^2}\sqrt{(k - m\omega^2)^2 + (c\omega)^2}$$

분자·분모를 모두 피크값으로 쓰거나 모두 RMS로 쓰면 같은 결과다. 한쪽만 피크값이면 $\sqrt{2}$ 배의 오류가 생긴다. 고조파가 있다면 두 값 모두 구동 주파수의 기본파 성분이어야 한다.

감쇠를 생략한 이전 식과의 관계

$$c = 0: \quad |\widehat{K_t}| = \frac{a_{\text{rms}}}{I_{\text{coil, rms}}\omega^2}|k - m\omega^2|$$

> c=0, ω=√(k/m)에서는 유한한 정현파 정상상태를 가정할 수 없다. 이 점에 유한한 측정값을 넣어 Kt=0이라고 결론 내리면 안 된다.

09. 무엇을 Kt라고 부를 수 있는가

식의 정확성과 실제 장치에서의 식별 가능성은 따로 확인해야 한다.

① 전자기 가진력과 흉벽 접촉력은 다른 양이다

$F_{em}=Kt \cdot i$는 코일과 자계가 만드는 전자기력 관계다. 내부 이동질량, 하우징, 패널, 패드, 흉벽이 모두 같은 변위로 움직인다는 보장은 없다. 패널 또는 흉벽 가속도에 임의의 총질량을 곱해 코일 고유 Kt를 얻을 수는 없다.

> 첨부 식을 쓰려면 x가 무엇의 변위인지 먼저 정해야 한다. 그 x에 대응하는 유효 m,c,k와 입력 힘의 관계가 확인되어야 한다. 코드의 값은 이 가정 아래의 모델 기반 추정값이다.

② m,c,k와 Kt를 전부 미지수로 두면 절대값이 정해지지 않는다

$X/I=Kt/(ms^2+cs+k)$에서 m,c,k,Kt를 같은 양수 α배 해도 전달함수는 같다. 따라서 운동/전류 응답만으로 네 값의 절대 크기를 동시에 유일하게 결정할 수 없다. 질량 등 독립적인 물리 척도나 교정 정보가 필요하다.

③ 자기 데이터로 만든 Kt를 같은 데이터의 정규화에 쓰지 않는다

$$\widehat{K_t} = \frac{a}{I\omega^2}|D| \quad \Rightarrow \quad \frac{a}{\widehat{K_t}I} = \frac{\omega^2}{|D|}$$

여기서 $D=k-m\omega^2+jc\omega$이다. 왼쪽 가속도·전류가 소거되어 오른쪽에 이미 가정한 모델만 남는다. 이 결과로 모델 또는 공진을 독립 검증했다고 말할 수 없다. 코드도 이 추정값으로 후보 순위를 바꾸지 않는다.

> 독립적으로 교정한 일정 Kt와 실제 코일 전류가 있으면 전자기 입력력의 크기를 추정할 수 있다. 그래도 흉벽 전달력이나 완전한 복소 FRF를 얻으려면 추가적인 전달 모델·위상·힘 검증이 필요하다.

10. 추가한 명령과 실제 사용 방법

Arduino IDE / ESP32 + Adafruit BNO08x / Serial 115200 / 줄바꿈(Newline)

명령	기능
P,m,c,k	m [kg], c [N·s/m], k [N/m] 설정
P 또는 P,CLEAR	현재 모델 확인 / 모델 미설정 복귀
M,f	기존처럼 가속도 측정; Kt 계산 생략
M,f,I	해당 시험의 코일 기본파 RMS 전류 [A] 연결
K,f,a,I	기존 기본파 RMS 자료로 수동 Kt 계산
L / S / C / X	로그 / 예비 후보 / 표 초기화 / 계측만 중단

위 m,c,k,f,a,I는 자리에 넣을 실제 숫자를 뜻한다. M은 가진기를 구동하지 않는다. 외부 구동부를 먼저 안정화하고 전류를 독립 계측한다. 전류가 변하면 같은 정상상태 구간에서 전류·가속도를 수집한 뒤 K 명령으로 계산한다.

계산 확인용 예시: 실측·구동 설정값이 아닌 합성 데이터

```
P,0.2,2,7000
K,30,9.072229145495438,0.05
```

예상 |Kt| = 2.000000 N/A, 모델 힘 RMS = 0.100000 N.
위 숫자는 코드 검산용이다. 사용자의 장치값으로 저장하거나 구동값으로 사용하지 않는다. 실제 m,c,k,I가 없으면 실제 Kt 수치를 계산할 수 없다.

기존 45 Hz, 6.307 m/s² 변환표는 기존 문서의 RMS 가정을 이어받았다. 이 값이 피크 또는 전체대역 RMS라면 기본파 RMS인지 먼저 확인해야 한다.

11. 출력·검증 범위와 출처

기존 DATA·LOG 형식과 속도 기반 예비 후보 판정을 유지했다.

출력과 오류 처리

KT_EST는 출처(BNO_FIT / MANUAL_UNCHECKED), 주파수, 가속도 RMS, 코일 전류 RMS, m,c,k, 모델 힘 RMS, |Kt|, 상태를 출력한다. 수동 K 입력은 센서 품질 검사를 수행하지 않는다. 자동 M 출력은 기존 샘플·회귀 품질 조건을 통과했을 때만 계산한다.

> MODEL_ESTIMATE_NOT_CALIBRATED = 모델 기반 추정, 교정 완료 아님.
> MODEL_UNSET / INVALID_INPUT / MODEL_NEAR_SINGULAR = 값 사용 금지.
> 미설정·불량 조건은 숫자를 만들어 채우지 않고 nan과 상태를 출력한다.

코드 검증 결과와 남은 범위

호스트 C++11 시험: 알려진 Kt 복원, RMS/피크 일관성, 무감쇠 특이점 차단, 잘못된 입력 거부, 전류값의 측정 간 재사용 방지, 과긴 명령 폐기, 기존 기본파 회귀·로그 계산을 통과했다.
실제 ESP32 코어/Adafruit 라이브러리로 한 보드 빌드와 센서 실측은 미실시다. SDA/SCL, 센서 법선 축, 실제 보고율, 진폭 정확도, 안정화 시간을 장치에서 확인해야 한다. 20~45 Hz는 코드 설정 범위이며 검증 완료 표시가 아니다.

자료 출처와 적용 범위

[1] 사용자 첨부 손필기, 기본 폴더.pdf (1쪽). 유도식 검토의 직접 원자료.

[2] 기존 로그스케일 PDF 및 BNO085 전체 측정 코드. 앞 6쪽을 유지·보완.

[3] H2W Technologies, Voice Coil Actuators. 힘-전류 비례 특성.

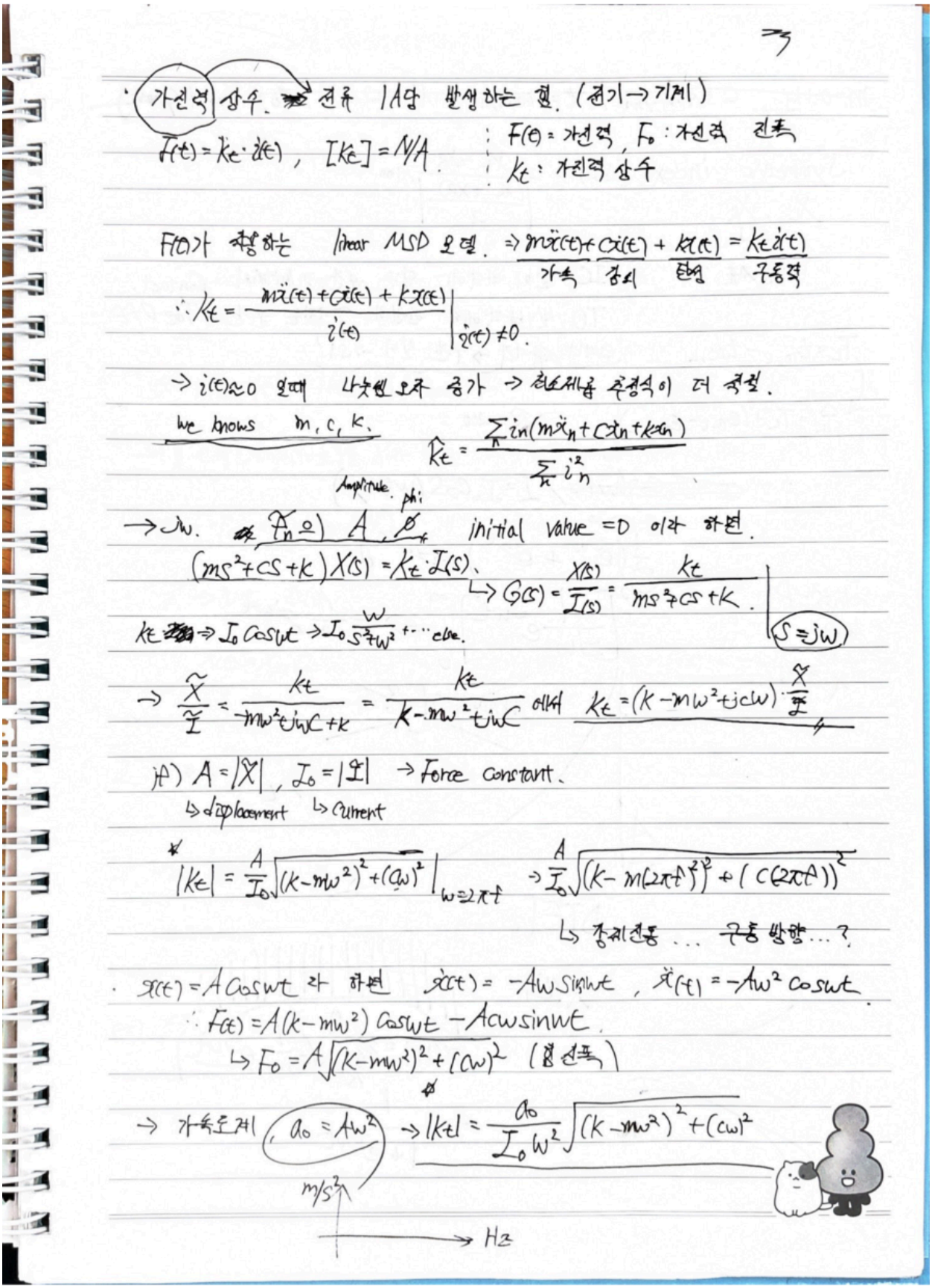
[4] TI, INA226 datasheet. 전류 계측 위치·변환·평균 설정 확인 자료.

[5] Adafruit BNO08x: sh2_SensorValue.h. timestamp 및 센서 구조.

웹 자료 확인: 2026.11.03. 주파수 식·식별성·순환 정규화 설명은 명시한 모델에서 직접 유도했다. INA226 공급전류가 코일 기본파 전류와 다르다는 것은 계측 위치에 따른 구분이다.

12. 첨부 필기 원문

유도식의 원문을 포함했다. 첨부에는 m,c,k,I의 숫자와 최종 Kt 수치가 없다.



부록 | 전체 Arduino 측정 코드

526행 전체 수록 · 코드 1/10쪽 · > 표시는 인쇄 줄바꿈, 소스의 새 줄이 아님

```
/*
 * 숨결톡톡 | Arduino IDE (ESP32 + BNO085) 공진 후보 측정 예제
 * 2026-11-03 / 로그 스케일+속도 보정+Kt 모델 추정 통합 측정 코드
 * 중요: 사용자 고유 구동 스케치가 아니며 구동 기능이 없는 측정 전용 예제
 *
 * 목적: 기존 웹 그래프 X=Frequency(Hz), Y=Acceleration(m/s^2) 유지
 *  - 종전의 전체 RMS 대신 구동 주파수 f 성분의 가속도 RMS 추출
 *  - 고주파 끝점 최대를 공진으로 자동 확정하지 않음
 *  - 내부 최대가 발견되면 '예비 후보'만 출력 (입력 가진력 미계측)
 *
 * 중요: 현재 구동 회로/핀/음원 PWM 코드가 제공되지 않았으므로
 *       이 예제는 트랜스듀서를 구동하거나 출력 설정을 변경하지 않는다.
 *       반드시 기존 구동 코드에서 주파수를 먼저 설정하고 안전 확인 후
 *       'M,<실제 구동 주파수>' 명령을 보낼 것.
 *       구동 제어를 통합하려면 해당 장치에 맞는 명령 확인/안정화/정지 로직 필요.
 *
 * 의존성: Arduino IDE, ESP32 core, Adafruit BNO08x 라이브러리
 *  - 측정축은 흉벽에 수직인 축으로 설정 (X/Y/Z 확인 필수)
 *  - 캘리브레이티드 가속도(SH2_ACCELEROMETER) 사용.
 *  - b0(DC)를 회귀로 제거하므로 중력 DC 오프셋은 영향 제한.
 *  - 데이터 유효 대역 및 실제 보고율을 별도 검증.
 *
 * Kt 추가: 첨부 필기에는 숫자가 없으므로 m,c,k 기본값은 미설정(NAN).
 * 모델: m*x'' + c*x' + k*x = Kt*i, 동일 좌표/선형/정상상태 가정.
 * 코일 전류는 같은 구동 주파수의 기본파 RMS를 외부 계측해 입력한다.
 * INA226 앰프 전원 전류를 코일 전류로 대신 입력하면 안 된다.
 * 흉벽/패널 가속도로 추정한 값은 구동기 고유 Kt나 접촉력을 보증하지 않는다.
 * 이 Kt 추정값을 동일 데이터의 힘 정규화에 재사용하지 않는다.
 *
 * SERIAL: 115200
 * P,m_kg,c_Ns_per_m,k_N_per_m : 독립적으로 확인한 모델 입력
 * P       : 현재 모델 확인 / P,CLEAR : 모델 미설정으로 복귀
 * M,f,I   : 외부에서 확인한 기본파 코일 RMS 전류(A)를 해당 측정에 연결
 * K,f,a,I : 기존 기본파 RMS 가속도(m/s^2)와 전류(A)로 수동 계산
 * K/KT_EST 결과는 추정값이며 교정값으로 자동 등록하지 않는다.
 * M,30    : 기존 구동 장치를 30 Hz로 안정화한 후 측정 시작
 * RESULTS : DATA,frequencyHz,accelRmsMps2,displacementRmsMm,samples,rateHz,R2,quality
 * S       : 가속도->속도 보정 후 내부 피크 후보 탐색
 * L       : 저장된 각 주파수의 로그 스케일 및 속도 지표 출력
 * C       : 저장한 결과 초기화
 * X       : 측정 취소(트랜스듀서 정지는 기존 구동 코드에서 별도 처리!)
 */

#include <Arduino.h>
#include <Wire.h>
#include <Adafruit_BNO08x.h>
#include <math.h>
#include <stdlib.h>
#include <string.h>

static constexpr uint32_t SERIAL_BAUD = 115200;
static constexpr uint32_t SENSOR_INTERVAL_US = 4000; // 목표 250Hz, 실제값은 계측 확인
static constexpr uint32_t SETTLE_MS = 800; // 실험 조건에 따라 검증 필요
static constexpr uint32_t MEASURE_MS = 1600;
static constexpr size_t MAX_SAMPLES = 800;
```

부록 | 전체 Arduino 측정 코드

526행 전체 수록 · 코드 2/10쪽 · > 표시는 인쇄 줄바꿈, 소스의 새 줄이 아님

```
56      static constexpr size_t MAX_POINTS = 48;
57      static constexpr float MIN_F_HZ = 20.0f; // 기존 설정 탐색 범위; 장치에서 검증 완료되었다는 뜻은 아님
58      static constexpr float MAX_F_HZ = 45.0f;
59      static constexpr char MEASURE_AXIS = 'Z'; // 반드시 실제 흉벽 법선 축 확인
60      static constexpr double PI_D = 3.14159265358979323846;
61      // 로그 레벨은 반드시 참조량을 함께 기록한다.
62      static constexpr double ACC_REF_MPS2 = 1.0;    // 0 dB re 1 m/s^2
63      static constexpr double VEL_REF_MPS = 0.001;   // 0 dB re 1 mm/s
64      static constexpr double F_REF_HZ = 30.0;       // 주파수 환산 비교 기준
65
66      // 값은 자동 추정하지 않는다. P 명령으로 독립 확인값을 설정한다.
67      double modelMassKg = NAN;
68      double modelDampingNsPm = NAN;
69      double modelStiffnessNPm = NAN;
70      double pairedCoilRmsA = NAN; // 현재 측정 1회에만 유효, 이전 전류 재사용 금지
71      static constexpr double NUMERIC_MIN_CURRENT_A = 1.e-6;
72      // 위 임계값은 0 나눗셈 방지용이며 전류계의 정확도/검출한계가 아니다.
73
74      bool modelIsReady() {
75        return isfinite(modelMassKg) && modelMassKg > 0 &&
76          isfinite(modelDampingNsPm) && modelDampingNsPm >= 0 &&
77          isfinite(modelStiffnessNPm) && modelStiffnessNPm >= 0;
78      }
79
80      void printModel() {
81        Serial.println("MODEL_HEADER,m_kg,c_Ns_per_m,k_N_per_m,status");
82        Serial.print("MODEL,"); Serial.print(modelMassKg,9);
83        Serial.print(','); Serial.print(modelDampingNsPm,9);
84        Serial.print(','); Serial.print(modelStiffnessNPm,9);
85        Serial.print(','); Serial.println(modelIsReady() ? "USER_MODEL" : "UNSET");
86      }
87
88      // |Kt| = a_rms/(I_rms*w^2) * hypot(k-m*w^2,c*w).
89      // 힘은 모델 추정값이다. 동일 좌표, 단일 주파수, 정상상태에서만 적용.
90      double estimateKt(double f, double a, double current,
91                        double &force, const char *&status) {
92        force = NAN;
93        if (!modelIsReady()) { status="MODEL_UNSET"; return NAN; }
94        if (!isfinite(f) || f<=0 || !isfinite(a) || a<=0 ||
95            !isfinite(current) || current<=NUMERIC_MIN_CURRENT_A) {
96          status="INVALID_INPUT"; return NAN;
97        }
98        double w=2.0*PI_D*f;
99        double elastic=modelStiffnessNPm-modelMassKg*w*w;
100       double viscous=modelDampingNsPm*w;
101       double d=hypot(elastic,viscous);
102       double scale=fmax(fabs(modelStiffnessNPm),
103                         fmax(fabs(modelMassKg*w*w),fabs(viscous)));
104       if (!isfinite(d) || !isfinite(scale) || !(scale>0)) {
105         status="NUMERIC_RANGE"; return NAN;
106       }
107       if (d<=1.e-6*scale) { status="MODEL_NEAR_SINGULAR"; return NAN; }
108       double fModel=(a/(w*w))*d;
109       double kt=fModel/current;
110       if (!isfinite(kt) || kt<=0 || !isfinite(fModel) || fModel<=0) {
```

부록 | 전체 Arduino 측정 코드

526행 전체 수록 · 코드 3/10쪽 · > 표시는 인쇄 줄바꿈, 소스의 새 줄이 아님

```
111         status="NUMERIC_RANGE"; return NAN;
112       }
113       force=fModel;
114       status="MODEL_ESTIMATE_NOT_CALIBRATED";
115       return kt;
116     }
117
118     void printKtEstimate(double f, double a, double current, bool manual) {
119       double force;
120       const char *status;
121       double kt=estimateKt(f,a,current,force,status);
122       Serial.println("KT_HEADER,source,Hz,aFundRMS_mps2,iCoilFundRMS_A,"
123                      "m_kg,c_Ns_per_m,k_N_per_m,FmodelRMS_N,KtAbs_N_per_A,status");
124       Serial.print("KT_EST,"); Serial.print(manual ? "MANUAL_UNCHECKED" : "BNO_FIT");
125       Serial.print(','); Serial.print(f,6);
126       Serial.print(','); Serial.print(a,9);
127       Serial.print(','); Serial.print(current,9);
128       Serial.print(','); Serial.print(modelMassKg,9);
129       Serial.print(','); Serial.print(modelDampingNsPm,9);
130       Serial.print(','); Serial.print(modelStiffnessNPm,9);
131       Serial.print(','); Serial.print(force,9);
132       Serial.print(','); Serial.print(kt,9);
133       Serial.print(','); Serial.println(status);
134     }
135
136     // 빈 항목, 초과 항목, NaN/Inf를 거부한다.
137     bool parseNumbers(const char *text, double *v, size_t required) {
138       for (size_t i=0;i<required;i++) {
139         char *end=nullptr;
140         v[i]=strtod(text,&end);
141         if (end==text || !isfinite(v[i])) return false;
142         if (i+1<required) {
143           if (*end!=',') return false;
144           text=end+1;
145         } else if (*end!='\0') return false;
146       }
147       return true;
148     }
149
150     // 아래 계산은 단일 정현파의 가속도 RMS에만 적용한다.
151     double velocityRms(double aRms, double fHz) {
152       if (!isfinite(aRms) || !isfinite(fHz) || aRms <= 0 || fHz <= 0) return NAN;
153       return aRms / (2.0 * PI_D * fHz);
154     }
155
156     double accelDb(double aRms) {
157       if (!isfinite(aRms) || aRms <= 0) return NAN;
158       return 20.0 * log10(aRms / ACC_REF_MPS2);
159     }
160
161     double velocityDb(double aRms, double fHz) {
162       double v = velocityRms(aRms, fHz);
163       if (!isfinite(v) || v <= 0) return NAN;
164       return 20.0 * log10(v / VEL_REF_MPS);
165     }
```

부록 | 전체 Arduino 측정 코드

526행 전체 수록 · 코드 4/10쪽 · > 표시는 인쇄 줄바꿈, 소스의 새 줄이 아님

```
166
167   // 가속도를 기준 주파수 F_REF_HZ로 환산한 보정 로그 레벨.
168   // v = a/(2*pi*f)에 고정 상수를 곱한 것이므로 피크 위치는 velocityDb와 같다.
169   double correctedDb(double aRms, double fHz) {
170     if (!isfinite(fHz) || fHz <= 0) return NAN;
171     double aLevel = accelDb(aRms);
172     if (!isfinite(aLevel)) return NAN;
173     return aLevel - 20.0 * log10(fHz / F_REF_HZ);
174   }
175
176
177   Adafruit_BNO08x bno08x(-1);
178   sh2_SensorValue_t sensorValue;
179
180   enum Phase { IDLE, SETTLING, CAPTURING };
181   Phase phase = IDLE;
182   uint32_t phaseStartMs = 0;
183   float targetHz = 0.0f;
184
185   struct Sample { uint64_t timeUs; float a; };
186   Sample samples[MAX_SAMPLES];
187   size_t sampleCount = 0;
188   uint64_t lastTimeUs = 0;
189   uint32_t nonmonotonicTime = 0;
190   uint32_t droppedEvents = 0;
191   uint8_t prevSequence = 0;
192   bool hasPrevSequence = false;
193
194   struct Point {
195     float hz;
196     float meanRms;
197     float sumRms;
198     float sumRmsSq;
199     uint8_t trials;
200   };
201   Point points[MAX_POINTS];
202   size_t pointCount = 0;
203
204   bool getAxisAcceleration(const sh2_SensorValue_t &ev, float &out) {
205     if (ev.sensorId != SH2_ACCELEROMETER) return false;
206     switch (MEASURE_AXIS) {
207       case 'X': out = ev.un.accelerometer.x; break;
208       case 'Y': out = ev.un.accelerometer.y; break;
209       case 'Z': out = ev.un.accelerometer.z; break;
210       default: return false;
211     }
212     return isfinite(out);
213   }
214
215   // 3x3 연립방정식을 부분 피벗팅 가우스 소거로 풀이.
216   // x(t)=b0+bC*cos(2*pi*f*t)+bS*sin(2*pi*f*t)
217   bool solve3x3(double aug[3][4], double x[3]) {
218     for (int k = 0; k < 3; ++k) {
219       int best = k;
220       for (int r = k + 1; r < 3; ++r)
```

부록 | 전체 Arduino 측정 코드

526행 전체 수록 · 코드 5/10쪽 · > 표시는 인쇄 줄바꿈, 소스의 새 줄이 아님

```
      if (fabs(aug[r][k]) > fabs(aug[best][k])) best = r;
    if (fabs(aug[best][k]) < 1.e-9) return false;
    for (int j = k; j < 4; ++j) {
      double v = aug[k][j]; aug[k][j] = aug[best][j]; aug[best][j] = v;
    }
    double d = aug[k][k];
    for (int j = k; j < 4; ++j) aug[k][j] /= d;
    for (int r = 0; r < 3; ++r) if (r != k) {
      double q = aug[r][k];
      for (int j = k; j < 4; ++j) aug[r][j] -= q * aug[k][j];
    }
  }
  for (int k = 0; k < 3; ++k) x[k] = aug[k][3];
  return true;
}

struct FitResult {
  bool ok;
  double rms;
  double r2;
  double sampleRate;
  double displacementMm;
};

FitResult fitFrequency(float f); // Arduino 자동 프로토타입의 사용자 타입 문제 방지
FitResult fitFrequency(float f) {
  FitResult result = {false, NAN, NAN, NAN, NAN};
  if (sampleCount < 64 || !(f > 0)) return result;
  uint64_t t0 = samples[0].timeUs;
  uint64_t t1 = samples[sampleCount-1].timeUs;
  if (t1 <= t0) return result;
  double dt = ((double)(t1 - t0)) * 1.e-6;
  if (dt < 1.0) return result;
  result.sampleRate = (sampleCount - 1) / dt;
  double sum[3][3] = {{0}}, rhs[3] = {0};
  for (size_t i = 0; i < sampleCount; ++i) {
    double t = ((double)(samples[i].timeUs - t0))*1.e-6;
    double ang = 2.0 * PI_D * (double)f * t;
    double x[3] = {1.0, cos(ang), sin(ang)};
    for (int r=0; r<3; ++r) {
      rhs[r] += x[r] * samples[i].a;
      for (int c=0; c<3; ++c) sum[r][c] += x[r]*x[c];
    }
  }
  double aug[3][4];
  for (int r=0; r<3; ++r) {
    for (int c=0; c<3; ++c) aug[r][c] = sum[r][c];
    aug[r][3] = rhs[r];
  }
  double b[3];
  if (!solve3x3(aug,b)) return result;
  double mean = 0;
  for (size_t i=0;i<sampleCount;i++) mean += samples[i].a;
  mean /= sampleCount;
  double sse=0,sst=0;
```

부록 | 전체 Arduino 측정 코드

526행 전체 수록 · 코드 6/10쪽 · > 표시는 인쇄 줄바꿈, 소스의 새 줄이 아님

```
276     for (size_t i=0;i<sampleCount;i++) {
277       double t=((double)(samples[i].timeUs-t0))*1.e-6;
278       double ang=2.0*PI_D*(double)f*t;
279       double fitted=b[0]+b[1]*cos(ang)+b[2]*sin(ang);
280       double e=(double)samples[i].a-fitted;
281       sse+=e*e;
282       double centered=(double)samples[i].a-mean;
283       sst+=centered*centered;
284     }
285     result.r2 = sst > 1.e-12 ? 1.0 - sse/sst : NAN;
286     double peak=sqrt(b[1]*b[1]+b[2]*b[2]);
287     result.rms=peak/sqrt(2.0);
288     result.displacementMm=1000.0*result.rms/pow(2.0*PI_D*f,2);
289     result.ok=isfinite(result.rms) && isfinite(result.r2);
290     return result;
291   }
292
293   void addPoint(float f, float rms) {
294     size_t at=pointCount;
295     for (size_t i=0;i<pointCount;++i)
296       if (fabs(points[i].hz-f)<0.01f) {at=i;break;}
297     if (at==pointCount) {
298       if (pointCount>=MAX_POINTS) { Serial.println("ERR,POINT_TABLE_FULL"); return; }
299       points[at]={f,0,0,0,0}; pointCount++;
300     }
301     Point &p=points[at];
302     if (p.trials==255) { Serial.println("ERR,TRIAL_LIMIT");return; }
303     p.sumRms+=rms; p.sumRmsSq+=rms*rms; p.trials++;
304     p.meanRms=p.sumRms/p.trials;
305   }
306
307   void reportFit() {
308     FitResult r=fitFrequency(targetHz);
309     bool rateOK = r.ok && r.sampleRate >= 5.0*targetHz;
310     bool timeOK = (nonmonotonicTime==0);
311     bool fitOK = r.ok && r.r2>=0.60; // 연구용 예시, 보편적 인증 기준 아님
312     bool valid = rateOK && timeOK && fitOK && droppedEvents==0;
313     const char *quality = !r.ok ? "INVALID_FIT" :
314        !timeOK ? "BAD_TIMESTAMP" :
315        !rateOK ? "LOW_SAMPLE_RATE" :
316        droppedEvents ? "DROPPED_SAMPLES" :
317        !fitOK ? "LOW_FIT_R2" : "MEASURE_OK";
318     Serial.print("DATA,"); Serial.print(targetHz,3);
319     Serial.print(','); Serial.print(r.rms,6);
320     Serial.print(','); Serial.print(r.displacementMm,6);
321     Serial.print(','); Serial.print(sampleCount);
322     Serial.print(','); Serial.print(r.sampleRate,2);
323     Serial.print(','); Serial.print(r.r2,3);
324     Serial.print(','); Serial.println(quality);
325     // 결과가 유효할 때만 '예비 후보' 판정에 포함
326     if (valid) addPoint(targetHz,(float)r.rms);
327     if (isfinite(pairedCoilRmsA)) {
328       if (valid) printKtEstimate(targetHz,r.rms,pairedCoilRmsA,false);
329       else Serial.println("KT_SKIP,INVALID_ACCELERATION_MEASUREMENT");
330     }
```

부록 | 전체 Arduino 측정 코드

526행 전체 수록 · 코드 7/10쪽 · > 표시는 인쇄 줄바꿈, 소스의 새 줄이 아님

```
331       pairedCoilRmsA=NAN;
332       phase=IDLE;
333       Serial.println("STATE,IDLE");
334     }
335
336     // 로그 스케일 출력은 별도 명령(L)에서만 수행해 기존 DATA 포맷을 깨지 않는다.
337     void printLogTable() {
338       if(pointCount==0) { Serial.println("LOG,NO_POINTS"); return; }
339       Point arr[MAX_POINTS];
340       memcpy(arr, points, sizeof(Point)*pointCount);
341       for(size_t i=0;i<pointCount;i++)
342         for(size_t j=i+1;j<pointCount;j++)
343           if(arr[j].hz<arr[i].hz) { Point t=arr[i]; arr[i]=arr[j]; arr[j]=t; }
344       Serial.println("LOG_HEADER,Hz,AccelRMS_mps2,Accel_dB_re_1mps2,VelocityRMS_mmps,Velocity_dB_re_1mmps,Corrected_dB_re_30Hz
>           ,Trials");
345       for(size_t i=0;i<pointCount;i++) {
346         const Point &p=arr[i];
347         double v=velocityRms(p.meanRms,p.hz);
348         Serial.print("LOG,"); Serial.print(p.hz,3);
349         Serial.print(','); Serial.print(p.meanRms,6);
350         Serial.print(','); Serial.print(accelDb(p.meanRms),3);
351         Serial.print(','); Serial.print(v*1000.0,6);
352         Serial.print(','); Serial.print(velocityDb(p.meanRms,p.hz),3);
353         Serial.print(','); Serial.print(correctedDb(p.meanRms,p.hz),3);
354         Serial.print(','); Serial.println(p.trials);
355       }
356     }
357
358     void printCandidates() {
359       if (pointCount<3) {Serial.println("SWEEP,INSUFFICIENT_POINTS"); return;}
360       Point arr[MAX_POINTS];
361       memcpy(arr,points,sizeof(Point)*pointCount);
362       for (size_t i=0;i<pointCount;i++)
363         for (size_t j=i+1;j<pointCount;j++) if(arr[j].hz<arr[i].hz) {
364           Point tmp=arr[i];arr[i]=arr[j];arr[j]=tmp;
365         }
366       // 기존과 달리 가속도 절대 최대가 아니라 속도 환산값으로 판정한다.
367       int best=0;
368       for (size_t i=1;i<pointCount;i++)
369         if(velocityRms(arr[i].meanRms,arr[i].hz) >
370            velocityRms(arr[best].meanRms,arr[best].hz)) best=(int)i;
371       if(best==0 || best==(int)pointCount-1)
372         Serial.println("SWEEP,BOUNDARY_MAX_NOT_RESONANCE");
373       int count=0;
374       for(size_t i=1;i+1<pointCount;i++) {
375         Point &p=arr[i];
376         double vHere=velocityRms(p.meanRms,p.hz);
377         double vLeft=velocityRms(arr[i-1].meanRms,arr[i-1].hz);
378         double vRight=velocityRms(arr[i+1].meanRms,arr[i+1].hz);
379         if(!(isfinite(vHere) && isfinite(vLeft) && isfinite(vRight)) ||
380            vHere<=vLeft || vHere<=vRight) continue;
381         double cv=INFINITY;
382         if(p.trials>=3 && p.meanRms>1e-9) {
383           double var=((double)p.sumRmsSq/p.trials-p.meanRms*p.meanRms);
384           cv=sqrt(fmax(0.0,var))/p.meanRms;
```

부록 | 전체 Arduino 측정 코드

526행 전체 수록 · 코드 8/10쪽 · > 표시는 인쇄 줄바꿈, 소스의 새 줄이 아님

```
      }
      Serial.print("CANDIDATE,");Serial.print(p.hz,3);
      Serial.print(',');Serial.print(p.meanRms,6);
      Serial.print(',');Serial.print(p.trials);
      Serial.print(',');Serial.print(cv,3);
      Serial.print(',');Serial.println(cv <= 0.15 ? "REPEATABLE_PRELIMINARY" : "NEEDS_REPEAT");
      Serial.print("V_SCORE,"); Serial.print(p.hz,3);
      Serial.print(',');Serial.print(vHere*1000.0,5);  // mm/s
      Serial.print(',');Serial.println(correctedDb(p.meanRms,p.hz),3);
      count++;
    }
    if(count==0) Serial.println("SWEEP,NO_INTERIOR_PEAK");
    // '공진 확정' 문구를 절대 출력하지 않는다.
  }

  void startMeasure(float hz, double coilRms) {
    if(phase!=IDLE) { Serial.println("ERR,BUSY");return; }
    if(!isfinite(hz) || hz<MIN_F_HZ || hz>MAX_F_HZ) {
      Serial.println("ERR,FREQUENCY_OUT_OF_CONFIGURED_RANGE");return;
    }
    targetHz=hz; pairedCoilRmsA=coilRms;
    sampleCount=0; lastTimeUs=0; nonmonotonicTime=0;
    droppedEvents=0; hasPrevSequence=false;
    phase=SETTLING; phaseStartMs=millis();
    Serial.print("STATE,SETTLING,");Serial.println(targetHz,3);
    // 트랜스듀서는 이 펌웨어가 제어하지 않음. 호스트/기존 펌웨어에서 설정 필수!
  }

  char commandBuf[160]={0};
  size_t commandLen=0;
  bool commandOverflow=false;
  void dispatchCommand(char *line) {
    if(strcmp(line,"X")==0){
      phase=IDLE; pairedCoilRmsA=NAN;
      Serial.println("STATE,MEASUREMENT_ABORTED_ONLY"); return;
    }
    if(phase!=IDLE){Serial.println("ERR,BUSY");return;}
    if(strcmp(line,"S")==0){printCandidates();return;}
    if(strcmp(line,"L")==0){printLogTable();return;}
    if(strcmp(line,"C")==0){
      pointCount=0; Serial.println("STATE,CLEARED"); return;
    }
    if(strcmp(line,"P")==0){printModel();return;}
    if(strcmp(line,"P,CLEAR")==0){
      modelMassKg=modelDampingNsPm=modelStiffnessNPm=NAN;
      printModel();return;
    }
    double v[3];
    if(line[0]=='P' && line[1]==','){
      if(!parseNumbers(line+2,v,3) || v[0]<=0 || v[1]<0 || v[2]<0){
        Serial.println("ERR,BAD_MODEL_EXPECT_m_kg_c_Ns_per_m_k_N_per_m");return;
      }
      modelMassKg=v[0]; modelDampingNsPm=v[1]; modelStiffnessNPm=v[2];
      printModel();return;
    }
```

부록 | 전체 Arduino 측정 코드

526행 전체 수록 · 코드 9/10쪽 · > 표시는 인쇄 줄바꿈, 소스의 새 줄이 아님

```
440     if(line[0]=='K' && line[1]==','){
441       if(!parseNumbers(line+2,v,3)){
442         Serial.println("ERR,BAD_K_EXPECT_Hz_aFundRMS_iCoilFundRMS");return;
443       }
444       printKtEstimate(v[0],v[1],v[2],true);return;
445     }
446     if(line[0]=='M' && line[1]==','){
447       bool hasCurrent=(strchr(line+2,',')!=nullptr);
448       if(!parseNumbers(line+2,v,hasCurrent?2:1) ||
449          (hasCurrent && v[1]<=NUMERIC_MIN_CURRENT_A)){
450         Serial.println("ERR,BAD_M_EXPECT_Hz_OPTIONAL_iCoilFundRMS");return;
451       }
452       startMeasure((float)v[0],hasCurrent?v[1]:NAN);return;
453     }
454     Serial.println("ERR,UNKNOWN_COMMAND");
455   }
456
457   void handleSerial() {
458     while(Serial.available()) {
459       char c=(char)Serial.read();
460       if(c=='\r') continue;
461       if(c=='\n') {
462         if(!commandOverflow && commandLen>0) {
463           commandBuf[commandLen]='\0'; dispatchCommand(commandBuf);
464         }
465         commandLen=0;commandOverflow=false;
466       } else if(!commandOverflow) {
467         if(commandLen+1<sizeof(commandBuf)) commandBuf[commandLen++]=c;
468         else {
469           commandOverflow=true;commandLen=0;
470           Serial.println("ERR,COMMAND_TOO_LONG_DISCARD_TO_NEWLINE");
471         }
472       }
473     }
474   }
475
476   void setup() {
477     Serial.begin(SERIAL_BAUD);
478     Wire.begin(); // 사용 중인 ESP32의 실제 SDA/SCL 핀 설정 유지/확인
479     Wire.setClock(400000); // 기판/배선 조건에 따라 변경하고 실제 샘플링 확인
480     if (!bno08x.begin_I2C()) {
481       Serial.println("ERR,BNO085_NOT_FOUND");
482       while(1) {delay(1000);}
483     }
484     if (!bno08x.enableReport(SH2_ACCELEROMETER,SENSOR_INTERVAL_US)) {
485       Serial.println("ERR,CANNOT_ENABLE_ACCELEROMETER");
486       while(1) {delay(1000);}
487     }
488     Serial.println("READY,NO_ACTUATOR_CONTROL,M-S-L-C-X-P-K-COMMANDS");
489   }
490
491   void loop() {
492     handleSerial();
493     if(bno08x.wasReset()) {
494       phase=IDLE;pointCount=0;pairedCoilRmsA=NAN;
```

부록 | 전체 Arduino 측정 코드

526행 전체 수록 · 코드 10/10쪽 · > 표시는 인쇄 줄바꿈, 소스의 새 줄이 아님

```
    if(!bno08x.enableReport(SH2_ACCELEROMETER,SENSOR_INTERVAL_US))
      Serial.println("ERR,BNO085_RESET_REPORT_FAILURE");
    else Serial.println("WARN,BNO085_RESET_SWEEP_CLEARED");
  }

  // 측정 중 데이터가 뒤늦게 들어오는 것을 방지하기 위해 정착 단계에서도 읽어 버림.
  for(int i=0;i<12 && bno08x.getSensorEvent(&sensorValue);++i) {
    float a=0;
    if(!getAxisAcceleration(sensorValue,a)) continue;
    if(phase!=CAPTURING) continue;
    if(sampleCount>=MAX_SAMPLES) {droppedEvents++;continue;}
    uint64_t t=sensorValue.timestamp;
    if(t==0 || (sampleCount>0 && t<=lastTimeUs)) {
      nonmonotonicTime++;continue;
    }
    if(hasPrevSequence) {
      uint8_t diff=(uint8_t)(sensorValue.sequence-prevSequence);
      if(diff!=1) droppedEvents+=(diff>0?diff-1:1);
    }
    prevSequence=sensorValue.sequence;hasPrevSequence=true;
    samples[sampleCount++]={t,a};lastTimeUs=t;
  }

  uint32_t now=millis();
  if(phase==SETTLING && (uint32_t)(now-phaseStartMs)>=SETTLE_MS) {
    phase=CAPTURING;phaseStartMs=now;
    sampleCount=0;lastTimeUs=0;nonmonotonicTime=0;
    droppedEvents=0;hasPrevSequence=false;
    Serial.println("STATE,CAPTURING");
  }
  if(phase==CAPTURING && (uint32_t)(now-phaseStartMs)>=MEASURE_MS) reportFit();
}
```